

CVM++ Report

Architecture, Design Rationale, Limitations, and Futureworks

Abhishek Tyagi and Anirudha Pratap Singh

May 18, 2026

Contents

1	Executive Summary	3
2	Repository Scope and Current Capabilities	3
2.1	Implemented Language Features	3
2.2	Tooling in the Repository	3
3	Architecture Deep Dive	4
3.1	Pipeline Overview	4
3.2	Frontend: Lexer	4
3.3	Frontend: Parser and AST	4
3.4	Backend: Compiler to Bytecode	5
3.5	Execution Layer: Stack-based VM	5
4	Architectural Design Choices and Rationale	6
4.1	Choice 1: Explicit IR Boundary (AST before Bytecode)	6
4.2	Choice 2: Stack Machine Instead of Register VM	6
4.3	Choice 3: 16-bit Operand Width	6
4.4	Choice 4: Separate Visualizer Binary	6
4.5	Choice 5: Example-driven Validation Strategy	6
5	Codebase Limitations	6
5.1	Language-Level Limitations	6
5.2	Parser and Diagnostics Limitations	7
5.3	Compiler and VM Limitations	7
5.4	Tooling and Operational Limitations	7
6	Risk Assessment	7
6.1	Near-Term Risks	7
6.2	Medium-Term Risks	7
6.3	Long-Term Risks	7
7	Future Work Roadmap	8
7.1	Phase 1: Correctness and Diagnostics Hardening	8
7.2	Phase 2: Testing and Developer Experience	8
7.3	Phase 3: Language Growth	8

7.4 Phase 4: Architecture Evolution	8
8 Illustrative Program and Stage Observability	8
9 Conclusion	9

1 Executive Summary

CVM++ is a compact custom-language implementation in C++ that demonstrates the full language-toolchain lifecycle: lexical analysis, syntax parsing, AST construction, bytecode generation, and stack-based virtual machine execution. The repository is intentionally lightweight and educational, emphasizing clarity and inspectability over feature breadth. This makes it highly suitable for compiler/VM learning, controlled experimentation, and incremental language extension.

The current codebase successfully executes a practical subset of imperative programming: variable declarations and assignment, arithmetic, conditional control flow, loops, boolean literals, and terminal input. The project also includes a dedicated visualizer executable that makes internal execution stages observable, improving debugging quality and pedagogical value.

The primary strengths of this repository are: (1) clean stage separation, (2) understandable control-flow lowering into bytecode, (3) transparent VM semantics, and (4) approachable extension pathways. The primary constraints are language incompleteness (notably missing comparison operators and string literals), limited diagnostics, and edge-case runtime semantics (for example chained assignment behavior).

From an engineering perspective, the repository represents a strong foundation for a teaching compiler and a viable base for a richer scripting runtime if systematic hardening is applied. This report details the architecture, design choices, known limitations, risk profile, and recommended future roadmap.

2 Repository Scope and Current Capabilities

2.1 Implemented Language Features

The implemented CVM++ subset currently supports:

- Numeric literals and unary minus.
- Arithmetic operators: `+`, `-`, `*`, `/`.
- Variable declaration and assignment using `let` and `=`.
- Boolean literals: `true`, `false`.
- Comparison operators: `<` and `==`.
- Control flow: `if/else` and `while`.
- Output: `print expression;`.
- Input primitive: `input`.

2.2 Tooling in the Repository

The repository contains two principal executables:

- **Runtime executable** (`cvm`): executes CVM++ programs from file or REPL.
- **Visualizer executable** (`cvm_visualiser`): shows stage-level internals (source, token stream, AST, symbol table, bytecode, VM output).

In addition, the repository includes curated successful and unsuccessful program suites under `examples/successful` and `examples/unsuccessful`, which are useful for regression checks and edge-case validation.

3 Architecture Deep Dive

3.1 Pipeline Overview

CVM++ follows a classic staged architecture:

1. **Lexing:** raw source text to token sequence.
2. **Parsing:** token sequence to abstract syntax tree (AST).
3. **Compilation:** AST to linear bytecode stream.
4. **Execution:** bytecode interpretation on a stack VM.

This decomposition intentionally isolates concerns. Each stage consumes a well-defined representation and emits a new representation with higher semantic structure or lower-level executability.

3.2 Frontend: Lexer

The lexer scans characters and emits typed tokens. It identifies punctuation, operators, keywords, identifiers, and integers. Unknown characters trigger an error print but do not necessarily abort lexing immediately.

Design benefit: the lexer is simple and approachable, minimizing accidental complexity.

Tradeoff: because errors are not represented as structured diagnostic objects, downstream parser errors can become secondary/follow-on symptoms rather than a single clean failure source.

3.3 Frontend: Parser and AST

The parser constructs an AST using recursive-descent style with precedence layering:

- assignment
- equality
- comparison
- term
- factor
- unary
- primary

Statement-level parsing supports declarations, print statements, expression statements, block statements, conditionals, and loops. AST node classes are explicitly modeled in `ast.h`, including leaf, unary, binary, and statement/control nodes.

Design benefit:

- Strong pedagogical clarity.
- Easy extension for new expression/statement forms.
- Bytecode compiler remains independent from concrete lexical grammar.

Tradeoff:

- Runtime error strings in `consume()` are currently generic and may not fully use context-specific messages.
- No recovery strategy (panic mode) means first parse failure generally terminates parse flow.

3.4 Backend: Compiler to Bytecode

The compiler traverses AST nodes and emits bytecode instructions with 16-bit operands where needed (values, variable IDs, jump targets). Control flow lowering uses patching of forward jumps:

- **if/else:** emit conditional jump placeholder, compile then-branch, optionally emit skip-else jump, then patch jump addresses.
- **while:** mark loop head index, compile condition, emit conditional exit jump placeholder, compile body, jump back, patch exit target.

A global symbol mapping (name to variable ID) is maintained across compilation contexts as needed.

Design benefit:

- Straightforward control-flow translation.
- Predictable and inspectable bytecode.
- Suitable for visualizer-driven debugging.

Tradeoff:

- Some index conversions trigger compile-time narrowing warnings (`size_t` to `int`).
- Dynamic-cast-heavy visitor approach may become harder to scale with larger AST surfaces.

3.5 Execution Layer: Stack-based VM

The VM interprets opcodes in a linear fetch-decode-execute loop. It maintains:

- Operand stack.
- Instruction pointer.
- Fixed-size variable memory (indexed by 16-bit variable IDs).

Arithmetic and comparison operations pop operands and push results. Branching instructions update instruction pointer based on condition and target addresses. Input reads from `stdin` and pushes numeric values to stack.

Design benefit:

- Minimal runtime footprint.
- Deterministic behavior and simple semantics.
- Easy to reason about for education.

Tradeoff:

- Limited type system (effectively integer + boolean-as-int convention).
- Input validation is shallow.
- Missing features in opcode set mirror language feature gaps.

4 Architectural Design Choices and Rationale

4.1 Choice 1: Explicit IR Boundary (AST before Bytecode)

The use of AST as intermediate representation is a deliberate choice to avoid direct source-to-bytecode translation. This improves maintainability and makes future optimization passes possible (constant folding, branch simplification, static diagnostics).

4.2 Choice 2: Stack Machine Instead of Register VM

A stack VM simplifies compiler implementation because expression trees map naturally to post-fix evaluation. It avoids register allocation complexity and is ideal for an educational first implementation.

4.3 Choice 3: 16-bit Operand Width

The transition from narrower operands to 16-bit operands extends representable constants, variable IDs, and jump destinations. This is a practical middle ground between toy-level constraints and unnecessary wide instruction encoding.

4.4 Choice 4: Separate Visualizer Binary

Keeping observability as a distinct executable (instead of embedding verbose diagnostics in runtime) is a professional choice that preserves clean runtime behavior while enabling deep introspection when needed.

4.5 Choice 5: Example-driven Validation Strategy

Curated successful and unsuccessful examples form a simple but effective regression framework. This pattern can evolve into automated CI-grade testing with expected outputs and expected-error snapshots.

5 Codebase Limitations

5.1 Language-Level Limitations

- Missing comparison operators: `>`, `<=`, `>=`, `!=`.

- No string literal support.
- No modulo operator, logical operators (**and/or/not**), or function abstraction.
- No arrays, maps, or user-defined composite types.

5.2 Parser and Diagnostics Limitations

- Error messages are often generic and can obscure root cause.
- Lack of token-span and caret diagnostics reduces usability for larger programs.
- No robust recovery path for multi-error reporting.

5.3 Compiler and VM Limitations

- Chained assignment case (for example `a = b = 7;`) can expose stack semantics mismatch and trigger stack underflow.
- Some indexing conversions emit warnings that should be addressed for production hardening.
- Memory model defaults can mask uninitialized-read intent in some flows.
- Input handling does not provide rich error recovery for invalid numeric input.

5.4 Tooling and Operational Limitations

- No formal unit-test harness integrated in build.
- No CI pipeline enforcement for warnings-as-errors or golden-output checks.
- Limited benchmarking and no formal performance baseline.

6 Risk Assessment

6.1 Near-Term Risks

- Feature additions may regress parser/compiler behavior without robust tests.
- Error-message quality may reduce adoption for non-developer users.

6.2 Medium-Term Risks

- As feature surface grows, dynamic-cast-heavy compilation and monolithic parsing logic can become difficult to evolve safely.
- Instruction encoding assumptions can become brittle without explicit versioning/spec documentation.

6.3 Long-Term Risks

- Without a typed value model, introducing strings/collections/functions may require disruptive VM refactors.
- Lack of formal language spec may lead to behavior drift across contributors.

7 Future Work Roadmap

7.1 Phase 1: Correctness and Diagnostics Hardening

1. Add missing comparison operators end-to-end (lexer/parser/compiler/VM).
2. Improve parser diagnostics with expected-token details and line context.
3. Resolve narrowing warnings and standardize index/offset types.
4. Add runtime checks for invalid input and clearer error propagation.

7.2 Phase 2: Testing and Developer Experience

1. Add automated regression tests based on existing example suites.
2. Add token/AST/bytecode golden tests for representative programs.
3. Integrate CI with deterministic build and test workflow.
4. Add failure snapshot tests for diagnostics quality.

7.3 Phase 3: Language Growth

1. Introduce string literals and print support for textual values.
2. Add logical operators and richer comparison suite.
3. Add modulo operator and optional short-circuit semantics.
4. Define minimal type tagging strategy for VM values.

7.4 Phase 4: Architecture Evolution

1. Formalize bytecode specification and versioning.
2. Evaluate typed value container (tagged union/variant) in VM.
3. Optionally introduce optimization pass layer between AST and bytecode.
4. Consider debug metadata mapping bytecode back to source lines.

8 Illustrative Program and Stage Observability

The visualizer demonstrates the key educational value of the codebase. For a non-trivial loop program:

Listing 1: Representative CVM++ Program

```
let n = 5;
let sum = 0;
while (0 < n) {
    sum = sum + n;
    n = n - 1;
}
print sum;
```


The visualizer can reveal:

- Full token stream for lexical correctness checks.
- AST structure for syntactic/precedence confirmation.
- Symbol table mapping for variable identity validation.
- Raw bytecode and disassembled instruction listing for backend verification.
- Final VM output for semantic end-to-end validation.

This observability pattern is a major strength and should be preserved as the language grows.

9 Conclusion

CVM++ is a well-structured educational compiler/VM project with clear separation of concerns and a practical observability toolchain. The implementation is already capable of executing meaningful imperative programs and demonstrates accurate lowering of control flow to bytecode.

The repository is currently best characterized as a *solid foundation* rather than a feature-complete runtime. Its strongest next steps are predictable: improve diagnostics, close feature gaps, formalize tests, and harden runtime semantics. With those steps, CVM++ can evolve from a learning-focused implementation into a robust mini-language platform suitable for assignments, demonstrations, and controlled scripting workloads.

Appendix: Suggested Build and Validation Commands

```
cmake -S . -B build
cmake --build build

# Runtime execution
.\build\Debug\cvm.exe .\examples\successful\10_while_countdown_sum.cvm

# Visualizer execution
.\build\Debug\cvm_visualiser.exe .\examples\successful\10_while_countdown_sum.cvm
```